

R. Poojitha
192311255

LO1-2

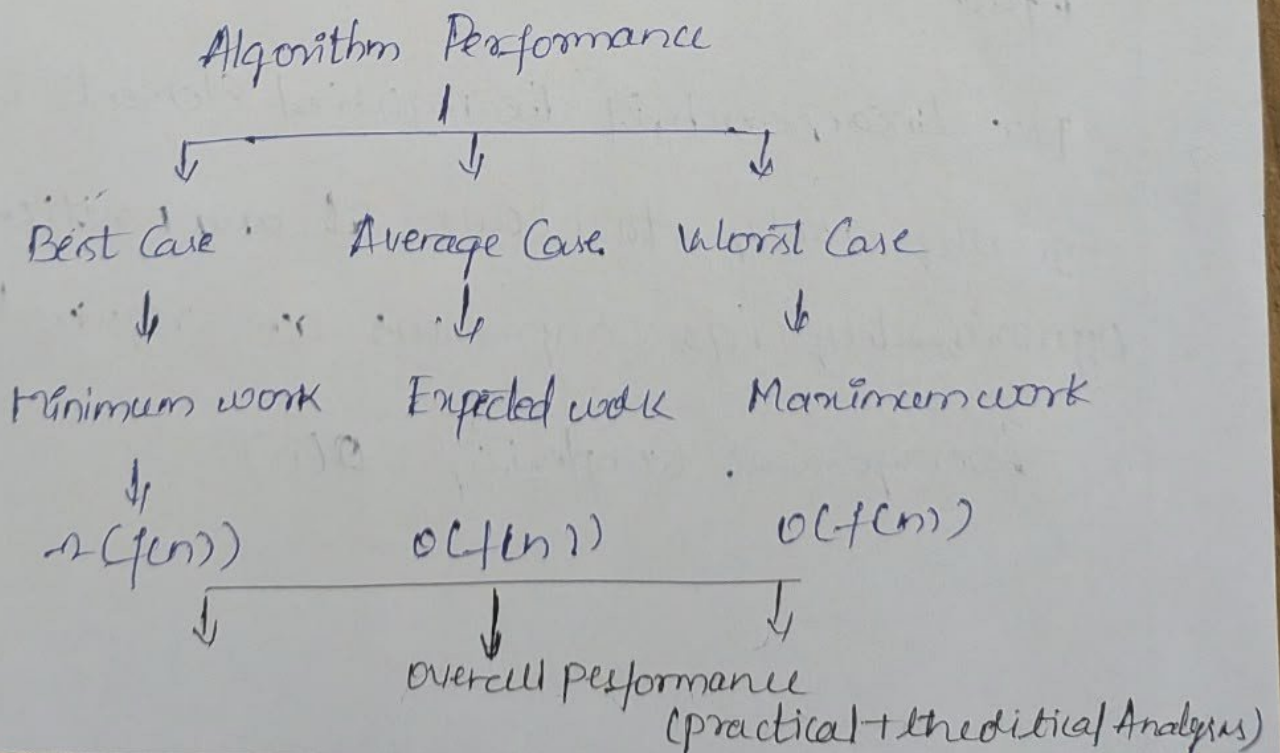
Relationship Between Best, Average and Worst Cases

The performance of an Algorithm can vary depending on the input given to it.

By Analyzing:

- (i) Best Case
- (ii) Average Case
- (iii) Worst Case

They can be connected to Asymptotic Notations such as Ω (Omega), Θ (theta) and O (Big-o)



Case	Meaning	Common Notation
Best Case	Minimum work	Ω (Omega)
Average Case	Expected / typical work	Θ (Theta)
Worst Case	Maximum work	O (Big O)

Ω - Lower Bound

Ω Describes a Lower Bound on Growth

$$T(n) = \Omega(1)$$

means the Algorithm requires at least constant order work in the Considered Analysis.

Θ - Tight Bound

Θ describes the tight Asymptotic Growth

$$T(n) = \Theta(n)$$

O - Upper Bound

O describes an upper bound on Growth

$$T(n) = O(n)$$

Means the running time does not grow faster than linear order, Asymptotically under the started Analysis.

Best Alone is insufficient

An algorithm might perform extremely well for a favourable input but poorly for most other inputs

Worst Case Alone is insufficient

might perform close to $O(n)$ for typical inputs

Average Case Alone is insufficient

Average Case Analysis on Assumptions about the Distribution of inputs.

Eg:

$$O(\log n) < O(n) < O(n \log n) < O(n^2)$$

practical Behaviour:

- * Input

- * Number of operations

- * Memory usage

- * Hardware, --- etc

Worst Case

The worst Case represents the Maximum Amount of work that an Algorithm may have to perform.

For Linear Search, if the Element is:

- * at the Last position or

- * not present at all

The Algorithm checks all n Elements

$\therefore$ Worst-Case Complexity $\geq O(n)$

Worst-Case Analysis is particularly important when we need to Guarantee that an Algorithm will not Exceed a Certain Amount of work.

Asymptotic Notation Describes how the running time grows as the Input size n becomes very large.

Best Case

The Best Case represents the Input for which the Algorithm performs the Minimum Amount of work

Example: Linear search

Array = [10, 20, 30, 40, 50]

Best-Case Complexity = $O(1)$

It Tells us how efficient the Algorithm can be under the most favourable Conditions

Average Case

The Average Case represents the Expected Performance over typical and randomly distributed inputs

For Linear search, if the required Element is Equally likely to occur at any position, Approximately $n/2$ Comparisons are required

Average-Case Complexity = $O(n)$